第六部分

强化学习推荐

第 28 章 强化学习推荐场景

在前面的章节中，我们围绕因果推荐展开了系统介绍，重点讨论了因果推荐的基本动机、反事实估计、因果工具以及因果推断技术在工业级推荐系统中的具体应用。本章开始进入强化学习（Reinforcement Learning, RL）推荐的部分，重点会给读者介绍强化学习技术在工业级推荐系统中的典型应用场景，方便不熟悉强化学习技术的读者从一开始建立起强化学习适用于哪些建模场景的整体认知。

与传统监督学习推荐模型相比，强化学习更加适合处理**序列决策（Sequential Decision Making）**的问题，通常我们会将整个问题建模为马尔科夫决策过程（Markov Decision Process, MDP），即序列决策中的相邻状态只依赖于前一个状态，具有马尔科夫性。在工业级推荐系统中，传统推荐模型通常只针对单次用户请求进行建模，比如给定当前用户的状态、候选物品以及上下文信息等，排序模型需要预测用户对物品的点击、观看、转化、互动等行为，然后推荐系统进一步通过排序模型的 pxtr 预估分来选择当前请求下的最优的物品推荐列表。这一类方法能够很好地优化单次曝光的即时收益，但其优化目标通常集中在当前请求，较难显式地描述当前决策对后续用户状态、后续请求以及长期用户价值产生的影响。

这种差异并不是简单的“模型是否使用了序列特征”的区别。很多传统推荐模型也会输入用户最近几十次甚至几百次行为，通过 Transformer、RNN 或者其他序列建模结构表示用户历史偏好，但这些模型最终学习的目标仍然通常是当前曝光对应的监督信号。例如，在精排阶段训练一个 CTR 模型时，即使输入包含完整的历史行为序列，优化目标依然主要是预测当前候选物品是否会被点击。换句话说，模型可以具有很强的历史信息建模能力，但并不意味着其已经成为一个真正意义上的序列决策模型。强化学习与这类方法的关键区别在于，强化学习不仅利用历史行为来预测当前 reward，还显式考虑当前动作（Action）对于未来状态以及未来收益的影响情况。

在真实的工业级推荐系统中，用户与系统之间并不是一次性的交互，而是由大量连续请求构成的动态过程。用户看到一个内容后可能会继续观看、跳过、点赞、评论、关注或者直接从 App 离开；而推荐系统则会根据用户最新的反馈行为来更新用户的状态，并在下一次请求中继续进行内容推荐。因此，一个在当前时刻看起来能够带来较高即时收益的推荐决策，往往并不一定能够带来更高的长期收益。例如，连续推荐大量高度相似的内容可能在当前请求中获得较高的点击率，但如果推荐的都是一些博眼球擦边的内容，长期会导致用户在后续请求中的兴趣快速衰减。除此之外，在短视频推荐中，如果为了追求短期的播放时长而持续提高某个品类内容的曝光占比，也可能会造成用户审美疲劳甚至长期留存下降。与此同时，推荐系统中的很多高层决策，比如不同内容类型之间的流量分配、探索与利用之间的平衡、不同阶段推荐策略之间的互相协同，本身也不太容易通过单次的监督学习来直接进行建模刻画。

进一步来看，推荐系统中的“长期价值”并不一定意味着非常长的时间跨度，比如周级别或者月级别的时间跨度。它可以是下一个请求、未来几个请求、一个 session，也可以是更长周期内的用户生命周期价值。因此，不同工业级推荐系统中的强化学习决策问题，它们的时间尺度可能并不相同。有些策略每个请求都需要执行一次，例如当前请求下是否要在最终的候选集中插入直播；有些策略可能每隔若干个请求调整一次，例如当前 session 中不同内容类型的流量比例；还有一些策略可能以天或者更长时间为决策周期，例如针对某类用户的长期探索强度。强化学习方法所提供的核心能力，是允许研究人员根据各自的业务问题来定义合适的状态、动作以及时间尺度，而不必将所有推荐问题都简化成单次的预测问题。

强化学习的核心优势正是在于能够将推荐系统描述为一个动态且连续交互的决策过程，通过状态（State）、动作（Action）、奖励（Reward）以及状态转移（State Transition）来刻画当前决策对未来用户行为的影响，并通过优化累计回报来学习长期策略。因此，在工业级推荐系统中，强化学习并不一定意味着“用一个 RL 模型直接替代现有的排序模型”，更多时候是作为传统推荐架构中的一个重要信息补充，用于解决传统监督学习无法解决或者比较难处理和建模的动态决策问题。

如果将推荐系统看成一个完整的反馈闭环，那么传统的预测模型与强化学习模型实际上承担的是两个不同的问题。传统预测模型通常更关注在给定状态下某个候选内容能够带来什么即时结果，例如点击概率、打赏概率、加购概率、观看概率等；而强化学习模型更加关注在多个可能的动作之间应该如何进行选择，以及当前状态下的动作选择对于未来状态以及未来状态下的累计收益会产生什么样的影响。因此，一个比较自然的工业级组

合方式是首先使用成熟的多任务学习模型来获得大量的基础预测信号，比如 CTR、CVR、时长、互动率等，再由强化学习模型在这些预测信号之上进行更高层的决策。

从工业落地角度来看，强化学习在推荐系统中的应用场景通常可以归纳为两种不同的层次。一类应用场景是链路维度下的**局部决策优化**，即在已有的召回、粗排、精排、重排或者混排模块体系中，引入强化学习模型来对某一类决策问题进一步优化，比如透出阶段调整不同内容的占比、排序阶段融合多个目标、重排混排阶段生成物品序列或者进行个性化探索等。另一类强化学习的应用场景是时间维度下的**跨请求长期优化**，即显式建模用户状态随连续请求的变化，使当前决策不仅考虑即时的 reward，同时也考虑未来一段时间内的累计长期价值。

除此之外，从决策的对象来看，工业级推荐中的强化学习也可以粗略地划分为三个层次。第一个层次是 Item 层的决策，即直接决定当前请求推荐哪个具体的 Item；第二层是 List 层或者 Slot 层决策，即决定一个推荐列表中不同位置放置什么内容以及不同内容之间如何组合；第三层是 Policy 层决策，即不直接选择具体的 Item，而是决定流量比例、融合权重、探索程度、策略切换以及推荐系统是否降级等更高层的策略参数。在实际的工业级推荐系统中，第三类场景往往更加容易与现有的推荐基础设施相结合，因为它可以最大程度地保留系统中原有的召回与排序能力，而让强化学习模型只聚焦于传统推荐系统中较难优化的策略问题。

需要注意的是，强化学习并不适用于所有的推荐问题。对于大量稳定、明确且能够通过监督学习直接建模的预测任务，例如 CTR、CVR、观看时长等预估任务，传统的监督学习模型仍然是工业级推荐系统的核心，比如粗排、精排阶段的主模型。在考虑是否需要用强化学习建模时，需要考虑以下问题：

笔记 当前场景是否存在连续决策、动作会影响未来状态、目标具有长期性、多个目标之间存在动态权衡，或者需要在线探索的问题。如果满足这些条件，那么强化学习才可能是一个合适的建模选择。

因此，工业级推荐系统中的强化学习技术往往以 Agent、策略层或者决策层的形式存在，它们与传统的推荐模型共同构成了完整的推荐模型体系。从另一个角度来看，如果一个问题只需要回答“这个 Item 好不好”，通常监督学习模型就已经能够解决大部分问题；而如果问题进一步变成“在多个可行方案中当前应该选择哪一个方案，并且这个选择会影响下一次决策”，那么强化学习的价值就会明显提高。例如，对单个 Item 预测观看概率属于典型监督学习问题，而在多个内容类型之间决定当前应该给直播多少曝光，则更接近强化学习问题；对一个候选 Item 预测即时时长属于预估问题，而决定当前推荐什么内容能够让整个 session 的累计观看时长最高，则更接近强化学习问题。通过上述这几个例子，相信读者已经对传统监督学习问题与强化学习问题的差异有了初步的理解。

本章暂时不对具体的强化学习算法展开深入讨论，而是从工业级推荐系统架构的角度，对强化学习较为典型的应用场景进行归纳。通过这些场景，读者可以先建立“强化学习究竟适合解决什么推荐问题”的整体认识；在此基础上，本书强化学习推荐部分的后续章节会分别针对不同的推荐场景介绍具体的 MDP 建模、状态与动作设计、奖励函数、策略学习、离线强化学习、在线探索以及工业部署等技术细节，从而形成从整体应用场景到具体建模方法再到工程落地细节的完整技术体系。

28.1 流量分配与调控

流量分配与调控是工业级推荐系统中非常典型的强化学习应用场景之一。对于复杂的信息流、直播、广告以及内容混排系统而言，推荐系统最终呈现给用户的内容通常并非来自单一的内容池，而是由多个不同的内容类型、业务频道或者流量来源共同竞争有限的曝光位置。例如，在同一次请求中可能同时存在普通消费短视频、直播内容、广告内容、电商内容、热点内容以及其他业务内容。不同内容类型的即时收益、用户价值和业务价值并不完全统一，因此工业级推荐系统需要在有限的曝光空间内持续进行多业务之间的动态权衡。

这一类问题的特殊之处在于，推荐系统真正需要决策的往往并不是某一个 Item 的精确排序分数，而是不同流量之间应该如何分配有限的曝光机会。例如，一个信息流请求中究竟应该有多少比例的直播内容、广告流量应该占据多大的比例、以及某个业务频道应该获得多少的流量预算，这些问题从本质上都属于资源分配问题。传统的排序模型通常负责在同一个内容池内学习不同 Item 之间的相对偏好，而流量调控则是进一步决定不同内容池之间的竞争关系。因此，这一层决策与底层物品级排序（Item-level ranking）在问题的结构上就存在着明显的差异。

传统的流量调控方式通常通过人工配置规则、固定比例或者简单的优化算法（比如 PID 控制）来实现。例如，在不同用户群体、不同时间段或者不同流量场景下，为某类内容设置固定的曝光比例或上下限。这种方式实现简单、可解释性强，但其策略通常较为静态。当用户兴趣、内容供给、业务目标以及系统整体流量结构发生变化时，固定比例很难持续保持最优。

除了固定比例之外，工业系统中还经常采用分桶策略，即根据用户类型、请求场景、地域、设备、时间段等维度预先划分若干流量桶，然后分别设置不同的内容比例。这类方法比全局固定比例更加灵活，但随着业务维度增加，人工规则很容易产生组合爆炸。一方面，某个桶中的最优比例并不意味着另一个相似桶也适用；另一方面，当用户状态连续变化时，离散化分桶本身也会损失大量状态信息。强化学习则可以将这种流量控制从静态配置进一步转化为基于状态的动态决策。

强化学习可以将流量分配问题转化为一个连续决策问题。强化学习 Agent 会根据当前推荐系统的状态来决定不同内容类型应该获得多少流量，然后推荐系统在执行这一动作之后产生用户行为反馈信号以及业务反馈结果，再由这些反馈信号去更新强化学习 Agent 的未来状态。比如，在混排的场景中，可以由强化学习策略来动态决定广告、直播以及普通内容之间的流量比例；在直播推荐场景中，可以根据当前用户的状态、直播供给的整体质量、当前用户的历史观看情况以及直播业务的整体约束，来动态调整直播内容在信息流中的曝光机会。对于直播推荐场景中的强化学习决策问题，状态通常同时具有用户侧与系统侧这两个维度。用户侧的状态可以包括用户近期行为、内容偏好、活跃程度、近期直播消费情况、当前 session 状态等；系统侧的状态则可能包含各类直播内容的供给量、实时流量、业务指标等。将二者结合之后，RL Agent 才能够判断“在当前用户和当前系统状态下，直播业务的流量应该如何分配”。图28.1给出了该场景下基于强化学习的流量分配与调控闭环，RL Agent 根据用户侧与系统侧状态动态决定各业务的流量配比，经约束修正后由混排系统执行，并通过用户反馈与业务指标构成的奖励持续优化策略。

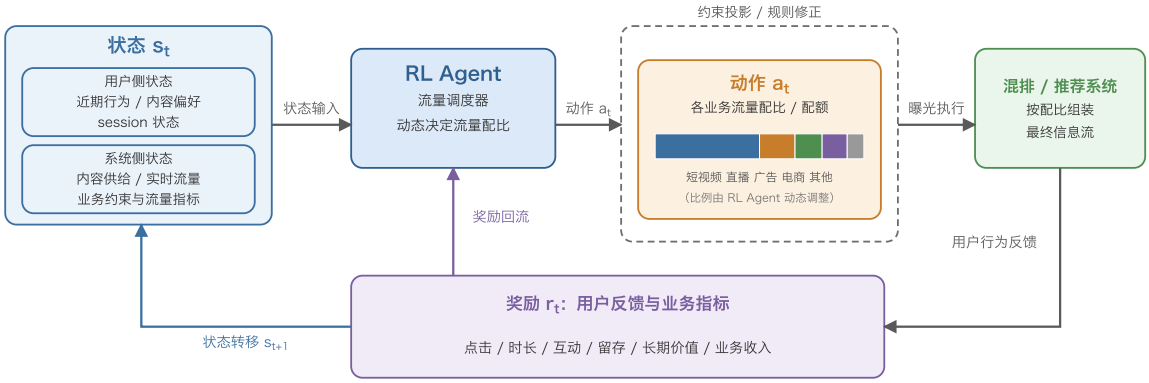


图 28.1: 基于强化学习的流量分配与调控闭环。

从 MDP 的角度来看，上述这些问题中的状态通常不仅包含单个用户的特征，还可能包含当前请求的上下文、近期用户行为、不同内容类型的历史曝光情况、内容供给状态以及系统级流量指标。动作则可以表示为某类内容的曝光比例、流量配额、插入概率或者多个业务流量之间的分配方案。而奖励既可以由点击、观看时长、转化等用户行为组成，也可以进一步加入留存、长期价值以及业务收益等指标。在动作空间设计上，不同的工业级推荐系统也存在明显差异。如果业务只涉及两种内容类型，那么动作可以简单地表示为一个连续比例；如果同时存在多种内容类型，则动作可能是一个高于 1 维的向量。在更加复杂的情况下，RL Agent 还可以选择一个流量档位，而不是直接输出精确的流量占比，从而降低线上 RL 策略的抖动与探索成本。因此，流量调控场景中的 RL 模型并不存在唯一的动作定义，真正重要的是动作空间是否与业务可执行的策略空间一致。

这种 RL 建模方式最大的优势在于：流量调控不再被视为一个静态比例配置问题，而能够被视为一个**动态资源分配问题**。某一类内容在当前时刻增加曝光后，不仅会影响当前请求的收益，还可能改变用户后续的状态以及当前内容与其他内容的竞争关系。强化学习可以通过长期回报来对这种未来的影响进行建模，从而学习到更加动态的流量调控策略。举个例子，当用户近期已经连续消费了多个直播内容，继续增加直播的曝光流量所带来的即时收益可能并不一定下降，但用户的长期满意度可能已经降低，因为在短视频流中插直播对于绝大多数

用户来讲都是一个兴趣打断的动作。因此，RL Agent 真正需要优化的并不是某一个固定比例，而是“在什么状态下应该使用什么样的流量配置”。同时，工业级流量调控通常还要满足非常强的业务约束。比如某些业务可能存在最小曝光量，某些业务的流量占比不能超过某个最大比例的阈值限制，广告或者商业化流量还可能受到广告主预算、库存等因素的限制。因此，RL 策略通常不能在完全自由的动作空间中进行探索，而需要进行带约束的 RL 建模，或者在 RL 输出之后再通过约束投影、规则修正等方式保证动作满足约束条件。这也是工业级强化学习与强化学习纯理论环境之间重要的一个区别。

从工业实践来看，强化学习尤其适合应用在混排、广告插入、直播流量控制以及不同业务流量协同等问题中。它的核心并不是简单地“让 RL 决定最终推荐什么”，而是让 RL 在传统推荐系统之上承担一个更加高层的流量资源调度器角色。在后续第29章节中，我们将会进一步讨论这类流量调控问题中的 MDP 定义、动作空间设计、长期奖励建模以及线上探索策略等内容。

28.2 流量降级优化

工业级推荐系统除了需要关注推荐质量，还必须保证推荐系统整体的稳定性。在极端流量、计算资源不足、依赖服务异常或者部分推荐模块故障时，推荐系统通常需要执行降级策略。例如，某些复杂模型可能被关闭，部分特征可能不再实时获取，某些请求可能切换到更轻量的推荐链路，或者直接采用缓存策略和兜底策略。与一般推荐任务相比，流量降级最大的特点是它同时受到用户体验约束和系统资源约束的影响。正常情况下，更复杂的推荐链路可能提供更高的推荐质量，但也会带来更高的计算和服务成本；而在系统负载升高时，继续使用完整链路可能造成整体服务不可用。因此，推荐系统真正需要优化的问题如下：

定义 28.1

在给定系统资源和稳定性约束下，如何通过动态调控是否进行流量降级来获得最大的推荐价值与收益。 ♣

传统的降级系统通常基于规则实现，例如当 CPU、GPU、延迟或者服务错误率超过某个固定阈值时，直接切换到预定义的降级方案。常见的降级方案包括 Redis 缓存降级，这种降级方案通常会在用户实时请求完整推荐链路时将候选集的一部分 Item 列表，比如 Top10 到 Top50 的 Item 统一缓存在 Redis 中，当需要流量降级的时候就自动从 Redis 中获取这些缓存好的 Item 列表给用户直接下发。这种基于固定阈值进行流量降级的方法通常具有较强的可控性，但只能根据当前系统状态进行局部决策，很难充分考虑降级策略对后续用户请求以及整体推荐效果的影响。例如，最简单的策略可能是在 CPU 超过某个阈值后直接关闭某个复杂模块。这种策略虽然容易实施，但实际上存在一个明显问题：推荐系统并不是只有“正常”和“不可用”两个状态。很多时候更合理的方式是在负载较高时逐步降低某些模块的计算复杂度，而不是一步切换到最激进的降级状态。如何在多个降级动作之间进行权衡，本身就是一个策略优化问题。

强化学习则可以将流量降级视为一个动态决策问题。RL Agent 可以根据当前系统负载、用户状态、请求特征、实时推荐质量以及历史降级情况等因素，来综合决定当前请求是否需要执行降级、以及应该采用何种降级策略。在这一场景中，状态通常需要比普通用户推荐问题更加丰富。除了用户的行为反馈之外，还可以加入 CPU、GPU、内存、网络、服务延迟、依赖系统状态、缓存命中率等系统指标；动作则可以表示采用正常链路、轻量链路、部分特征缺失链路或者完整降级链路。奖励则需要同时反映推荐质量和系统资源成本。图28.2给出了基于强化学习的流量降级决策过程，RL Agent 综合系统状态与请求特征在多种推荐链路之间动态选择，奖励同时刻画推荐质量与系统资源成本。

当前流量是否降级这个问题的一个重要特点是：降级请求本身会改变后续系统状态。在实时流量与降级流量同时存在的推荐系统中，如果相邻两个请求的处理方式不同，那么后一个请求所处的状态可能受到前一个请求的影响。例如，前一个请求执行降级后，用户没有获得正常推荐内容，其后续行为分布可能发生变化；同时，系统资源也可能因此得到释放，进而影响后续请求是否需要降级。因此，在构建 MDP 时，不能简单地把“当前请求是否降级”视为彼此独立的二分类问题，而需要考虑相邻请求之间的状态转移关系。以最简单的二值降级状

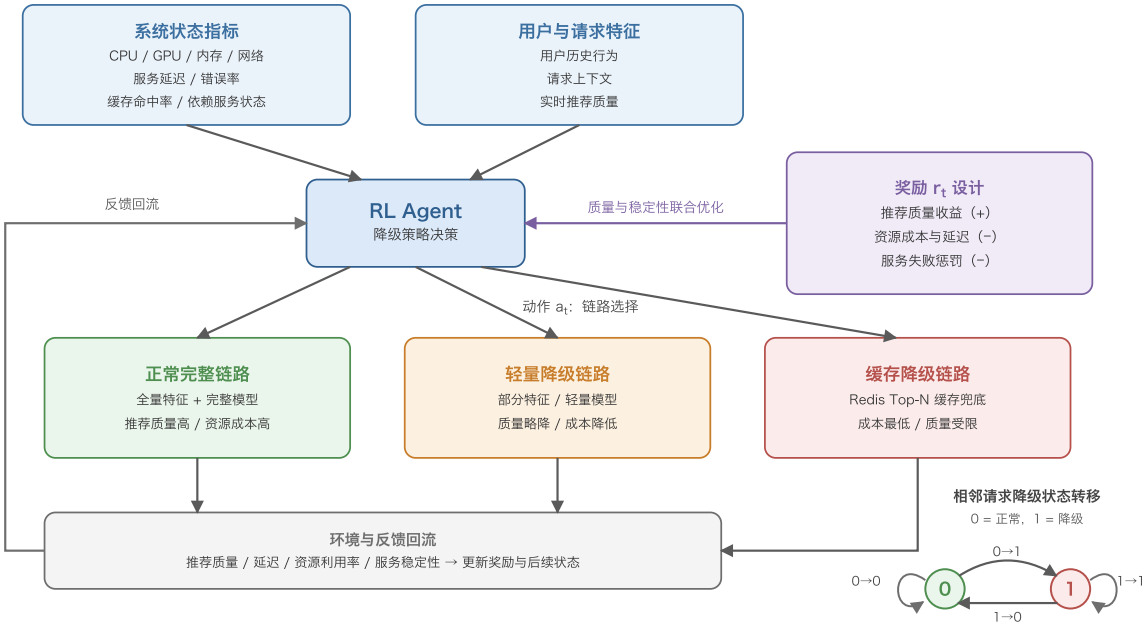


图 28.2: 基于强化学习的流量降级决策（右下角为相邻请求降级状态转移）。

态为例，相邻两个请求是否降级存在四种组合：

$$(0,0),\quad (0,1),\quad (1,0),\quad (1,1) \tag{28.1}$$

其中 0 表示正常流量，1 表示降级流量。不同组合对应不同的状态转移路径，因此策略设计需要显式考虑降级状态对于后续请求的影响。进一步来看，降级通常还具有一定的持续性。如果某个服务已经出现明显异常，那么连续几个请求采用不同降级状态可能具有不同的成本。相邻请求之间的相关性会使得问题更加接近真正的序列决策，而不是独立的分类问题。RL Agent 在这里的作用正是通过状态转移显式建模这种动态关系。

在流量降级优化这种场景下，奖励函数通常也需要同时考虑推荐效果和系统性能。一方面需要尽可能保持用户侧的观看、点击和长期价值；另一方面还需要满足延迟、资源利用率、服务稳定性等系统约束。因此，强化学习在这里承担的并不是单纯提升推荐指标，而是实现推荐质量与系统稳定性的联合优化。例如，可以将推荐质量收益看成正向 reward，将过高延迟、资源消耗或者服务失败作为负向成本，从而得到综合的优化目标。与简单的硬阈值规则相比，这种方式允许推荐系统在不同状态下寻找更加细粒度的折中。例如，在低负载情况下优先保障推荐质量，在高负载情况下主动牺牲部分复杂度换取整体稳定性。

此外，流量降级优化问题还具有一个比较明显的工业级特征，即系统安全性的优先级通常远高于探索的收益。推荐系统可以允许在某些用户流量上进行一定程度的探索，但系统降级策略通常不能承担过高的探索风险。因此，实际部署往往需要通过安全边界、动作白名单以及兜底策略约束 RL Agent，使策略只能够在可接受的操作空间内进行决策。这类问题也体现了强化学习在工业系统中的一个重要价值：它不仅能够优化“推荐什么”，还可以优化“在什么系统状态下应该采用什么推荐策略”。这也意味着强化学习的应用范围并不局限于内容推荐本身，还可以向推荐系统的基础设施与系统控制层进行延伸。在后续第29章节中，我们也会讨论工业界已落地的强化学习流量降级优化方案，并对方案落地中的难点和问题进行讨论。

28.3 多任务融合

工业级推荐系统通常不是单目标优化的系统，而是需要多目标联合优化的系统。以短视频、直播与信息流推荐为例，推荐系统同时需要关注点击、观看、有效观看、点赞、关注、转发、留存以及其他业务指标。大部分传统推荐系统都会通过多任务学习模型分别预测这些用户行为反馈的 label，然后再通过线性加权求和、乘法公式融合、LTR 模型或其他融合方式来得到最终的排序分数。

假设推荐系统中存在 M 个不同的预测目标，那么对于候选物品 i 而言，粗精排阶段主模型输出的多目标预

估值如下：

$$\mathbf{p}_i = [p_i^{(1)}, p_i^{(2)}, \cdots, p_i^{(M)}] \tag{28.2}$$

其中 $p_i^{(m)}$ 表示候选物品 i 在第 m 个预测目标上的预估值，$m = 1, 2, \cdots, M$。传统多任务融合方式往往进一步构造统一的排序分数，即：

$$s_i = f\left(p_i^{(1)}, p_i^{(2)}, \cdots, p_i^{(M)}\right) \tag{28.3}$$

其中 $f(\cdot)$ 可以是人工设计的线性加权、非线性融合模型或者学习得到的排序函数。但需要注意的是，传统多任务融合的最大问题之一是不同任务目标之间通常会存在明显的冲突或者业务夹角。例如，点击率高的内容不一定具有更高的观看时长，强互动内容不一定具有更高的长期价值；此外，平台的商业化目标与用户的消费体验目标也可能存在着直接竞争。因此，多任务模型虽然可以同时输出多个目标的预估结果，但“多个任务的预估结果究竟应该如何组合”仍然是一个独立的研究问题。

从工业级推荐系统的角度来看，在多任务融合任务中使用固定权重的融合方式实际上隐含了一个很强的假设，即所有用户以及所有场景都采用相同的多目标权衡方式。然而，真实的推荐系统通常并不存在一个在所有状态下都成立的最优多任务融合权重。例如，新用户、活跃用户以及低活用户可能具有不同的价值函数；工作日和休息日可能具有不同的用户消费模式；当前 session 刚开始的首刷与已经消费了较长时间的后续刷次之间通常也可能存在不同的目标偏好，在 session 中靠后刷次用户的消费耐心和投入度都会明显下降。

在多任务融合场景中，强化学习主要用于多任务融合的动态策略学习。一种非常直接的方式是将不同目标的融合权重作为 RL Agent 的动作，由 RL Agent 根据当前状态来动态决定各个任务目标的重要程度。RL 策略可以根据用户近期观看行为、session 状态以及当前内容供给情况来动态调整时长、互动与长期留存等目标的权衡比例。在这种情况下，传统的多任务预估模型仍然负责提供基础预估值，而 RL Agent 负责决定这些预估值之间如何进行组合。这样就可以将整个问题拆解成“预测”与“决策”两个层次。由于 CTR、观看时长等预估任务已经有大量成熟的工业级推荐模型，因此没有必要重新训练一个完全端到端的 RL 网络替代这些预测模块，而可以让 RL 模型更加聚焦于后续多任务融合策略学习的部分。与此同时，强化学习还可以进一步优化传统多目标融合中的超参数。工业级推荐系统中经常会存在大量需要人工调节的超参数，比如乘法公式融合中每个因子的缩放、偏置与幂次的超参数、不同内容类型之间的约束强度以及不同流量场景的融合系数等。强化学习可以将这些超参数的选择转化为策略优化问题，通过在线或离线反馈来自动搜索更优的参数组合，从而降低人工调参的成本。图28.3对比了传统固定权重融合与 RL 动态权重融合两种方案，传统方案对所有用户与场景使用同一组融合权重，而 RL Agent 根据当前状态动态输出融合权重，并通过包含长期价值的奖励回流持续优化融合策略。

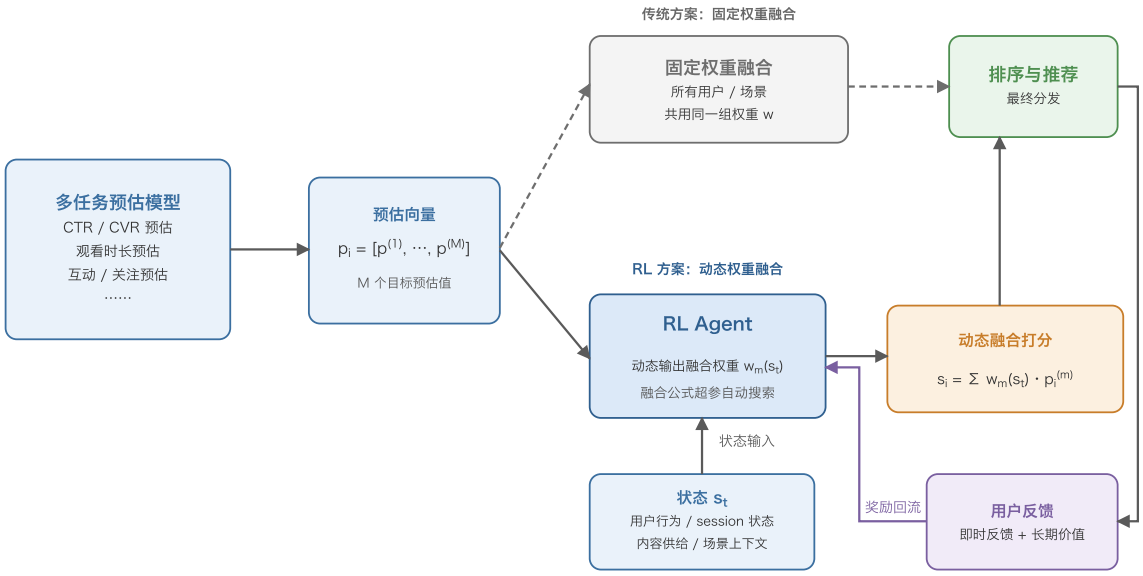


图 28.3: 固定权重融合与 RL 动态权重融合对比。

这一思路也可以扩展到更加复杂的策略参数。例如，某些场景中不同目标的权重并不是一个固定标量，而

可能依赖于用户群体、内容类型、位置甚至已经生成的推荐序列。此时，可以将原本复杂的 if-else 策略规则看做是一个待学习的策略函数，让 RL 模型从简单的“调一个参数”进一步发展为“根据状态动态生成策略参数”，或者让离线 RL 模型直接学习复杂策略的输出结果。更重要的是，多任务融合任务本身天然具有长期性。传统多任务融合主要优化当前请求下用户的即时反馈 reward，而强化学习可以进一步考虑某种融合策略对当前用户后续请求的影响。例如，一次过度偏向次均观看时长的决策可能提高当前刷次的 reward，但会降低用户后续活跃度和 session 内的消费深度；相反，一些当前即时收益不那么高但能够提升用户长期兴趣匹配的内容，可能具有更高的长期价值。因此，在 session 粒度或者更长的用户生命周期粒度上定义累计 reward，能够使 RL 模型学习到更加符合长期价值的多任务融合策略。

此外，这里还存在一个非常重要的问题，即多个任务之间的价值并不一定可以通过简单线性加权表达。假设当前请求的点击和时长同时上升，并不意味着二者带来的收益可以简单相加；而某些目标可能具有阈值效应或者边际递减的特性。因此，在 RL 建模的过程中，我们需要将最终的 reward 直接定义为业务层面的价值函数，而不是将多个预估目标机械地进行加和。这使得 RL 能够在更高层次上学习“哪些组合能够带来更高的整体价值”。在实际工业级推荐系统中，通常会保留成熟的 CTR、时长、转化等基础预估模型，再由强化学习策略对这些预估结果进行动态决策与长期价值微调。这种“预测模型负责刻画即时行为，RL 模型负责进行长期决策”的方式，更符合工业级推荐系统的模块化架构。

进一步来说，多任务 RL 融合还可以与传统 LTR 模型结合。例如，原有 LTR 模型继续负责学习不同候选物品之间的即时排序关系，而 RL 模型输出一个 session-level 的价值或 policy-level 的调节信号来对 LTR 分数进行修正。这种方式可以复用现有的工业级推荐系统基础设施，同时避免直接将整个排序链路转化为高成本的在线 RL 系统。在后续的第30章中，我们将会详细讨论多任务 RL 融合模型的具体实现与应用。

28.4 Session 级推荐优化

传统工业级推荐模型通常以单次请求作为主要的优化单位。无论是点击率、时长、互动等目标的预估模型，还是粗排、精排模块中的主模型、LTR 模型以及部分重排模型，大部分的训练目标都围绕当前曝光对应的 label 或者 reward 展开。因此，这类监督学习的模型通常擅长回答“当前请求推荐什么最可能获得即时收益”这个问题，但对于“当前推荐的内容会如何影响整个 session 的表现”这个问题往往缺乏显式的建模。

需要强调的是，即使当前请求输入了完整的历史行为序列，这也不意味着模型已经直接优化了 session 级别的目标。例如，一个模型利用了用户最近 100 次的交互行为来预测当前内容的观看概率，但这个模型还是会被看作是一个学习即时反馈信号的监督学习模型。只有当前 action 的价值不仅由当前 label 所决定，还与未来一系列状态以及 reward 发生关联时，这才真正属于序列决策的问题。实际上，用户在推荐系统中的行为通常具有较强的连续性。大部分用户在一次请求中观看了某类内容后，他下一次请求的兴趣状态会随之发生变化；而在连续观看大量相似的内容之后，用户也可能产生审美疲劳。因此，当前请求下的推荐结果不仅会决定当前的 reward，也会影响到用户后续请求的状态。

因此，Session-level RL 建模可以将用户的整个 session 建模成一个序列决策过程：

$$s_t \rightarrow a_t \rightarrow r_t \rightarrow s_{t+1} \tag{28.4}$$

其中 s_t 表示用户在当前请求时的 session 状态，a_t 表示当前推荐决策，r_t 表示该请求产生的即时 reward，而 s_{t+1} 则由当前推荐结果和用户行为共同决定。在实际工业系统中，s_t 往往不是某一个固定特征，而是一组综合状态。例如，它可以包含用户长期兴趣、近期兴趣、已经曝光的内容类型、最近连续跳过次数、session 已持续刷词数量、当前 session 已获得的累计 reward 等。这样可以让 RL 策略区分“刚进入 session 的用户”和“已经持续消费较长时间的用户”，并做出不同的决策。

与单次请求优化不同，Session 级 RL Agent 的优化目标不再仅仅是最大化当前请求的 reward，而是最大化整个 session 的累计收益，级：

$$R = \sum_{t=1}^{T} \gamma^{t-1} r_t \tag{28.5}$$

其中 γ 表示折扣因子。Session 级 reward 的设计可以根据具体的业务定义，比如累计观看时长、累计有效消费次数、session 留存、连续观看深度或者多种用户行为指标的组合。这里需要特别注意，reward 的定义并不等于某一个简单的行为 label。如果直接把每次点击作为 reward，那么 RL 模型最终大概率学出来的会是一个高度偏向提升点击率的策略。如果要真正体现 RL 模型的长期价值优化能力，则需要让 reward 设计能够表达清楚当前业务真正希望优化的目标。通过这种方式，RL Agent 可以主动学习当前决策与未来用户行为之间的关联，同时也可以平衡对于当前即时价值与未来长期价值的学习。当某个内容在当前请求下的即时价值略低，但用户观看该内容后更有可能继续浏览后续内容的时候，那么该内容可能具有更高的 session 级价值。在这种情况下，传统的即时排序模型通常可能因为其当前预估分数较低而降低曝光，相反 RL 模型则通过最大化未来累计 reward 识别到该内容的长期收益属性，因此会选择提高其曝光。图28.4将上述 session 级序列决策过程进行了直观刻画，整个 session 被建模为 $s_t \to a_t \to r_t \to s_{t+1}$ 的马尔可夫决策过程，优化目标为累计折扣回报，决策价值可分解为即时价值与长期价值两部分。

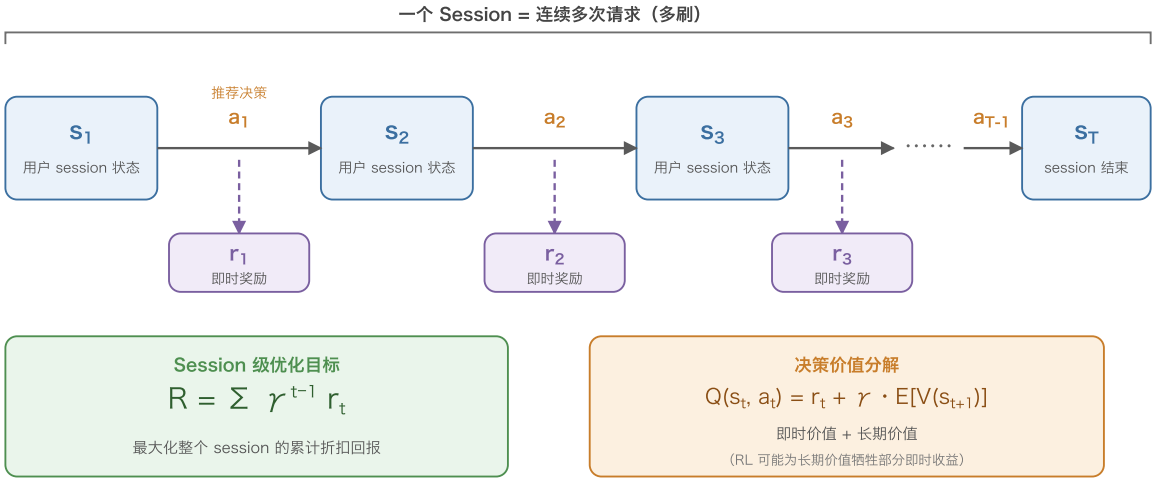


图 28.4: Session 级推荐优化的序列决策建模。

针对即时模型降低曝光而 RL 模型提高曝光这一现象，可以进一步从即时价值（Immediate Value）与长期价值（Long-term Value）的分解来理解其背后的差异。对于候选 Item i，当前时刻产生的即时 Reward 可以记为 $r_t(i)$，而真正需要关注的决策价值还应包括该 Action 对未来状态及后续 Reward 的影响，即：

$$Q(s_t, i) = r_t(i) + \gamma \mathbb{E}_{s_{t+1}} \left[V(s_{t+1}) \mid s_t, i \right] \tag{28.6}$$

其中，第一项 $r_t(i)$ 表示 Item 当前能够带来的即时收益，第二项则表示该 Item 对未来累计收益的贡献。传统排序模型通常更加关注当前 Reward，例如直接优化点击、观看时长等即时反馈；而 RL 模型则进一步考虑 Action 对未来状态的影响，试图在即时价值与长期价值之间进行权衡。因此，对于某些当前 Reward 较低、但能够改善后续用户状态并带来更高长期收益的 Item，RL 模型可能会给予其更高的选择概率，从而出现即时模型降低曝光、而 RL 模型反而提高曝光的现象。

Session 级 RL 模型还可以用于解决连续内容消费中的疲劳和兴趣漂移问题。例如，当用户连续消费同一垂类内容时，当前单个 Item 的点击率可能仍然较高，但继续推荐同类内容可能导致用户后续的 reward 持续下降。此时，Session 级 RL 模型可以通过 session 中的状态转移识别到这种变化趋势，从而主动在当前请求的候选集中引入不同类型的内容，使得在当前请求下略微牺牲即时收益，但能够换取更高的后续累计收益，保证 session 级整体收益的最大化。

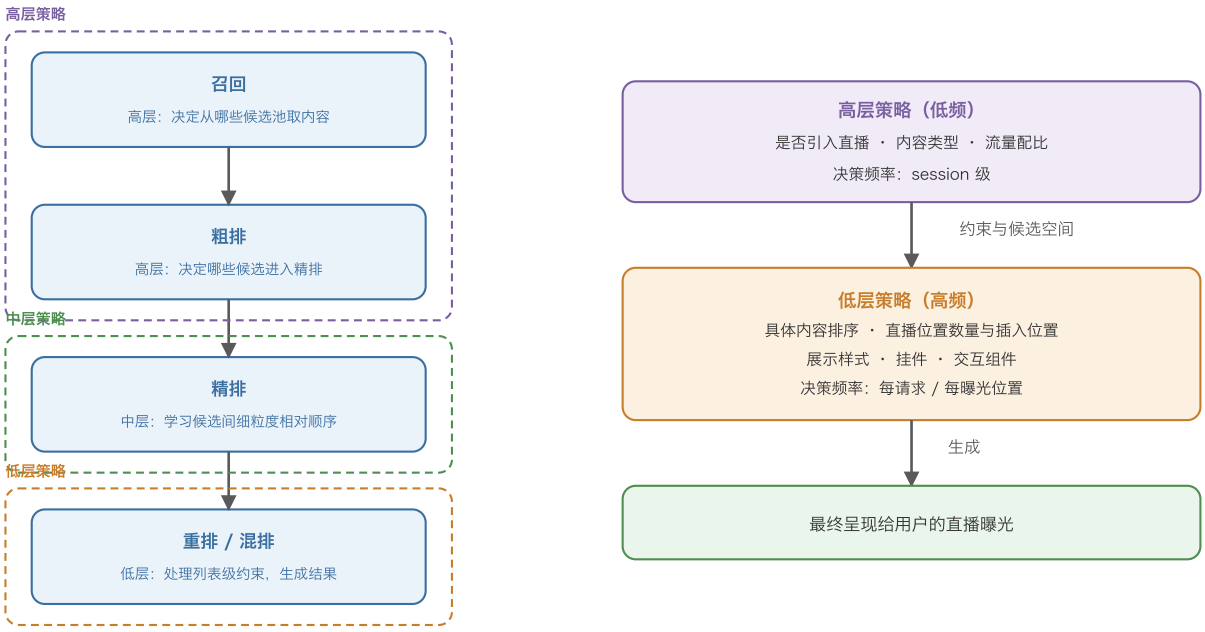
因此，Session 级 RL 建模的核心作用并不是简单替换原有排序模型，而是为传统排序系统引入一层长期价值优化的视角。排序模型仍然可以负责学习精细的内容偏好与用户的即时行为概率，而 Session 级 RL Agent 则负责判断不同内容在整个 session 中的长期价值，从而为当前排序提供额外的决策信号。在具体落地实现时，Session 级 RL 模型通常并不一定直接决定分发某个具体的 Item，它可以在推荐系统的排序环节输出一个 session 级的调整项。例如，在根据传统排序模型产生的 pxtr 进行多目标融合之后，RL 模型可以根据当前 session 的具体状态

给不同类型候选 Item 的融合分数增加或减少某个长期价值的修正项，当然也可以选择乘法或除法来实现。这种方式可以在保留原始排序机制的同时增加系统长期决策的能力，这也是工业界落地 session 级 RL 建模最经典的一种链路生效方式。在工业级推荐系统中，这类方法尤其适用于短视频、直播、信息流等推荐场景。后续第31章节将会进一步讨论 session 级推荐优化中的 state 构造、长期 reward 定义、序列状态建模等问题。

28.5 层次化决策

工业级推荐系统通常来说是一个非常复杂的系统。在级联式推荐架构中，一个完整的推荐请求可能经过内容召回、粗排、精排、重排、混排等多个阶段，而且每个阶段承担着不同粒度的决策任务。例如，高层决策可能决定当前请求应该推荐哪一类内容，低层决策则在确定的内容类型中进一步选择具体 Item。从业务视角下来看整个推荐问题，如果在一次推荐中既可以给用户推荐视频、也可以推荐新闻、广告等多种内容类型，那么推荐系统实际上需要同时解决内容类型的选择以及某个内容类型下 Item 的选择这两个不同层次的决策问题。因此，工业级推荐系统中的一些业务问题可以天然地抽象为**层次化决策（Hierarchical Decision Making）**的问题，其中高层策略负责决定“应该做什么”，低层策略负责决定“具体怎么做”。这种层次化建模的方式可以将复杂的推荐决策拆解为不同时间尺度以及不同粒度的策略优化（Policy Optimization）问题。

图28.5从两个视角给出了层次化决策的示意：(a) 全链路视角下，级联推荐链路中的召回、粗排、精排、重排/混排等阶段分别对应高层、中层与低层策略；(b) 产品视角下，以直播业务为例，高层策略决定是否引入直播以及流量配比，低层策略决定具体直播内容、位置数量以及展示样式、挂件、交互组件等产品决策。下面分别就这两个视角展开讨论。



(a) 全链路视角：级联推荐系统层次化建模　　(b) 产品视角：单业务决策的层次化扩展（直播为例）

图 28.5: 层次化决策的两种视角。

以级联式推荐系统架构为例，层次化决策与其天然存在的多阶段结构具有较高的契合性。一种理想化的方案，是将整个级联式推荐架构建模为一个层次化的 RL Agent：召回阶段的高层 RL Agent 决定从哪些候选池中获取内容，粗排阶段的高层 RL Agent 决定哪些候选内容值得进入精排，精排阶段的中层 RL Agent 进一步学习候选内容之间的细粒度相对顺序，而重排和混排阶段的低层 RL Agent 则负责处理列表级约束，最终生成推荐结果。可以看到，传统级联式推荐系统虽然本身具有明显的层次结构，但各阶段之间通常依赖预先定义的人工规则或固定排序策略进行衔接，而强化学习则提供了一种对不同层级推荐策略进行联合优化的思路。需要强调的是，由于强化学习通常存在训练样本效率较低、线上推理稳定性不足等问题，将整个级联式推荐架构全面改造为层次

化强化学习系统的方案，目前在工业界尚未形成大规模落地的实践先例。

以内容类型选择为例，高层 RL 策略可以决定当前请求下更适合推荐短视频、图文、新闻或者直播内容；低层 RL 策略则进一步在对应内容池中选择具体候选并进行精细排序。对于直播推荐而言，高层策略可以决定当前请求是否应该下发直播内容，低层策略则决定具体应该下发多少个直播位置、哪些直播内容或者这些直播应该插入信息流中的什么位置。在直播业务中，即使直播下发的位置数量已经固定，层次化决策依然可以继续向下扩展。例如，高层 RL Agent 可以决定在当前请求下是否引入直播，而低层 RL Agent 则决定直播内容给用户曝光展示时是否使用特定样式、是否展示挂件、是否采用某种交互组件或者其他展示策略。此时，不同策略并不一定直接负责选择直播内容，而是负责不同粒度的产品与推荐决策。像这种产品与推荐联动的层次化决策在直播推荐、增长 push 优化中都非常常见，在工业界也有一些落地应用的案例存在。

需要强调的是，上面这些例子中讲到的分层决策其实并不只是为了把一个大的动作拆解成两个小的动作，更重要的是不同的决策层级之间通常存在着不同的决策频率。例如，高层策略可能只需要在一个 session 开始时决定当前内容类型偏好，或者每隔若干请求调整一次流量配比；而低层策略则需要在每一个请求甚至每一个曝光位置上做出具体的推荐决策。因此，高层与低层策略实际上也可能分别对应不同时间尺度下的决策过程。

层次化 RL 的一个重要优势是可以降低单个 RL Agent 所承担的问题复杂度。如果让一个策略同时决定内容类型、候选 Item、展示位置以及业务样式，那么 RL Agent 要建模的动作空间就可能非常庞大，而且不同动作之间的语义差异也会很大。通过将决策过程拆解为多个层级，我们就可以分别建模高层策略与低层策略，从而提高 RL 模型离线训练与在线部署的可行性。此外，我们还可以给高层策略和低层策略分别设计不同的 reward。高层策略更加适合优化内容类型之间的长期价值以及业务级指标，而低层策略更加关注具体内容的消费效果。两者并不一定必须共享完全相同的 reward，可以根据不同业务的特点进行灵活的设计。这也给工业级推荐系统中的分层优化提供了更大的设计空间，比较考验算法工程师对于业务的思考与理解。

层次化决策通常能够较好地处理业务约束。例如，高层策略先确定当前允许使用的内容池以及最大流量比例，低层策略只能在这一可行空间内选择具体内容。这样可以显著缩小低层策略的动作空间，也可以避免 RL 模型直接探索到违反业务规则的动作。另一方面，层次策略之间也存在信用分配问题。例如，session 内整体的 reward 可能由高层 RL Agent 与低层 RL Agent 的多个决策共同产生，因此需要判断某次收益提升究竟来自于高层 RL Agent 的内容类型选择还是低层 RL Agent 的具体内容选择。

此外，层次化 RL 模型并不意味着一定需要多个完全独立的 RL Agent。在实际部署中，我们也可以使用统一的策略网络同时输出高层动作与低层动作，或者通过不同的策略头（policy head）来实现不同粒度的决策。至于选择哪一种架构，需要读者综合考虑状态空间、动作空间、线上推理耗时以及现有推荐系统的模块边界等方面来做出权衡。在工业级推荐系统中，层次化决策尤其适合内容类型选择、业务流量调度、复杂混排以及多阶段推荐等场景。后续第31章中，我们将会进一步详细介绍层次强化学习的状态划分、高层 RL Agent 与低层 RL Agent 如何协同等话题。

28.6 序列生成优化

在级联式推荐架构中，重排和混排模块通常处于推荐系统中较为靠后的决策阶段，它们的核心目标不再只是简单判断某个 Item 本身的价值，而是要从一组候选 Item 中构造最终呈现给用户的推荐序列。因此，这类任务天然具有序列决策特征。如果只对单个 Item 独立打分，那么最终结果通常由分数排序得到。但在实际的推荐列表中，不同位置之间通常来说并不是互相独立的。例如，前面已经出现多个相似内容时，下一个位置再推荐同类型内容的边际收益可能会显著降低；此外如果直播类型的内容连续出现，通常也可能违反“营收类内容不能连着出”的业务约束。因此，候选 Item 列表的价值实际上是由多个 Item 共同决定的。

在本书前面的第7章和第8章中，我们已经介绍了重排和混排模块的基本原理。传统重排或者混排方法通常采用逐位置打分、贪心选择、Beam Search、规则约束或者基于优化目标的组合算法等来生成候选的 Item 列表。虽然这些方法可以较好地处理多样性、内容比例以及业务约束，但往往仍然以相对局部的目标进行决策。例如，在选择当前第 k 个 Item 时，主要考虑候选 Item 当前的价值以及与已经选择内容之间的关系。从这个角度看，重排与混排问题实际上具有一种典型的组合优化特征。具体来说，假设目前有 N 个候选 Item，我们需要生成长度

为 K 的 Item 列表，那么可能的候选组合数量为 $\frac{N!}{(N-K)!}$，即组合数量会随着 N 和 K 的增大而指数级增长。传统基于规则以及基于贪心选择的序列生成策略虽然能够在工程落地中高效地运行，但通常很难直接优化最终序列的长期价值。因此，强化学习提供了一种非常自然的建模方式，即：

定义 28.2

强化学习序列生成问题可以形式化为一个马尔可夫决策过程（MDP），其中状态表示当前已经生成的序列以及用户和候选 Item 的特征，动作表示从剩余候选中选择下一个 Item，把每一步选取一个 Item 视为一次 action，随着序列逐步生成，状态不断更新。

所以，强化学习可以直接将“生成最终推荐序列”视为一个逐步决策过程。RL Agent 在每一步选择一个候选 Item，随着序列不断生成，系统状态也随之更新，直到生成完整推荐列表。此时，单个动作不仅决定当前位置展示什么内容，还会影响后续可以选择的内容以及整个列表的最终价值。在这一过程中，状态可以包含用户状态、候选 Item 的特征以及当前已经生成的部分列表；动作则是从剩余候选中选择下一个 Item；reward 可以在每一步即时反馈，也可以在完整序列生成后统一反馈。这样，RL 模型可以直接针对列表级指标进行优化，而不需要把列表目标拆解成多个彼此独立的 Item 级 label。此外，序列中的位置本身也存在价值的差异。第一个位置的曝光机会通常与最后一个位置不同，因此序列生成的策略不只是“选哪些 Item”，还会涉及到“哪些 Item 应该放在什么位置”。如果直接将每个位置的选择作为独立 action，那么在混排环节是可以处理的，而在重排环节会导致动作空间与状态空间快速膨胀；因此工业级 RL 模型通常也需要结合候选池的约束、动作分解或者其他方法的来控制建模的复杂度。图28.6给出了强化学习序列生成的逐步决策过程，RL Agent 每一步从剩余候选中选择一个 Item 追加到序列中，重复 K 步生成完整推荐列表，并以列表级指标作为奖励。

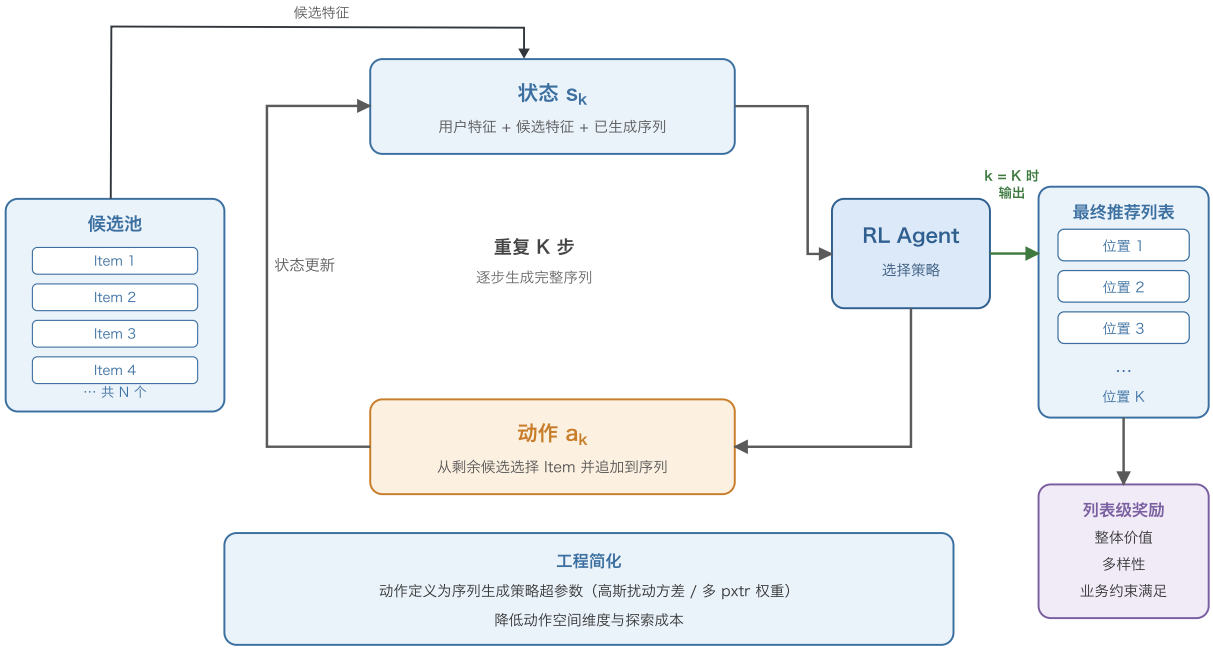


图 28.6: 强化学习序列生成的逐步决策过程。

但需要注意的是，如果把 action 定义为选择一个候选 Item，那么动作空间的规模将随着候选池的大小而迅速增长，尤其是在候选数量较大时，可能导致探索成本过高和训练不稳定的问题。除此之外，如果 action 是一个候选 Item，那么当前候选 Item 可能并不一定会曝光，reward 也无法使用其后验真实曝光的用户反馈作为 reward。因此，在实际应用中，通常并不会将序列生成 RL 模型的动作空间设计为直接选择单个候选 Item，而是会基于已有的序列生成策略来进行动作空间的设计，比如在多 pxtr 组合带高斯扰动的序列生成策略下，我们可以将 RL 模型的动作设计为高斯扰动的方差，或者包括多 pxtr 组合时的权重。通过控制序列生成策略超参数的方式来探索与生成最优的候选序列，而不是直接选择单个 Item。这样的好处是，可以显著降低动作空间的维度，从而减少探索成本，提高训练的稳定性，同时也能够更好地结合已有的序列生成策略，实现对最终推荐序列的有效优化。

前面提到的“最终序列 RL”和“序列生成 RL”这两种不同层次的建模方式在实际应用时还是存在一定的差别。前者更加接近最终页面展示，直接影响用户看到什么，比如在混排阶段候选 Item 比较少的时候可以使用；后者则处于更上游的位置，比如重排阶段，其目标是提高重排序阶段候选集合的整体质量与多样性。序列生成 RL 如果能够提前将具有长期价值的内容纳入候选集合，即使后续重排序列评估阶段不是使用 RL 模型，那么也可以相对有效地为这些内容提供进一步竞争与曝光的机会。

这种 Generator 级别的 RL 模型具有一个重要价值，即：如果某些内容本身在传统即时评分下并不占优势，但与其他内容组合后能够产生更高的序列价值或 session 价值，那么 RL Generator 也可以通过序列级的 reward 将这些内容纳入候选序列。这样，强化学习优化的不再只是“哪个 Item 分数最高”，而是“哪些内容组合在一起能够形成更高价值的推荐序列”。同时，Generator 层面的 RL 模型还能够承担序列生成时的探索作用。当序列生成阶段的大部分召回元都长期偏向某些熟悉的内容时，RL Generator 可以通过线上策略的探索将一些新的内容引入最终候选集合，从而降低整个推荐陷入信息茧房的问题。对于长尾内容、冷启动内容以及新业务类型，这种适度探索的机制尤其有意义。通常在冷启动业务优化的过程中，为了更好地提升新内容的曝光和获取用户反馈，在重排以及混排模块的序列生成阶段通常都会有的各自的 Generator 来保障最终的冷启曝光流量，同时在其他 Generator 中引入随机探索通常也会对冷启视频的曝光更加友好。

从更广泛的角度来看，序列生成 RL 模型可以同时解决三个层次的问题。第一，在 Item 层面优化用户兴趣的匹配；第二，在列表层面优化多样性、业务约束和多目标组合；第三，在更长时间尺度上优化用户的长期价值。因此，它是强化学习在推荐系统中非常重要的一类应用。当然，序列生成问题本身也是工业级 RL 方案落地应用中较为复杂的场景之一。由于动作空间通常非常大，候选数量可能达到数百甚至数千，如何设计动作空间、降低探索成本、处理离线数据中的支持集问题以及保证线上推理效率，都是后续第32章节中需要重点讨论的问题。

28.7 个性化探索

传统推荐系统通常更加偏向于利用已有用户兴趣，即根据历史行为预测用户最可能喜欢的内容，并优先推荐高置信度的 Item。这种“利用（Exploitation）”策略能够带来稳定的即时收益，但长期来看容易形成内容同质化以及用户兴趣收缩的问题。对于推荐系统而言，探索（Exploration）具有独立且重要的价值。一方面，新内容、新作者和长尾内容需要获得一定的曝光机会；另一方面，系统只有真正向用户展示新的内容，才能获取用户对于这些内容的新反馈，从而更新用户兴趣认知。因此，推荐系统本质上存在一个持续的探索与利用之间的权衡（Exploration-Exploitation Trade-off）的过程。

推荐系统中的探索并不是单纯为了“多推荐一些不确定的内容”，它的真正目的有两个：第一，通过探索发现用户原本未知的兴趣；第二，通过探索获取新的反馈信息，从而减少系统对用户兴趣认知的不确定性。这意味着某个候选内容即使当前预测 reward 并不高，只要它能够带来较高的信息增益，也可能具有一定的探索价值。

在传统推荐系统中，我们通常通过规则实现探索，例如给新内容设置固定流量、对不同垂类配置探索比例、设置随机曝光池或者对低曝光的 Item 进行适度的加权处理。这些方法实现起来比较简单，但通常难以做到真正的个性化。例如，一个已经表现出明确二次元内容兴趣的用户，随机推荐一个陌生垂类内容的探索价值可能相对有限，同时反而降低流量分发效率；而对于一个近期行为非常分散、兴趣模式不太稳定的用户，探索更多不同类型内容就显得更有价值一些。因此，真正合理的探索策略应该依赖于用户当前状态，而不是对所有用户使用相同的随机流量比例。

强化学习可以根据用户当前状态来动态决定探索的程度。例如，对于已经具有稳定兴趣分布的用户，可以提高高置信度内容的利用比例；对于用户兴趣尚不明确的新用户，则可以适当增加探索；对于长期只消费单一垂类的用户，也可以通过个性化策略逐步增加其他内容类型的探索机会。在这个过程中，RL Agent 的状态不仅可以包含用户历史兴趣，还可以包括近期已经曝光的内容类型、探索历史、用户对冷启内容的响应以及当前推荐系统对用户兴趣的不确定性等因素。而 RL Agent 的动作可以表示探索某类内容、某个新 Item 或某个垂类，也可以进一步表示探索的比例或者某类探索 Item 的个数。RL Agent 的奖励则不仅包含即时消费收益，还应该考虑探索带来的未来价值，例如后续兴趣发现、长期观看时长、用户满意度以及内容生态价值。这里可以将探索动作的价值理解为“当前收益 + 信息价值 + 未来收益”。传统的排序模型通常会更加关注前者，而 RL 模型可以通过

未来 reward 学习后两个部分的收益。这样，推荐系统就能够主动区分“虽然现在看起来一般，但可能值得尝试”和“当前完全没有价值，而且探索成本也很高”的候选内容。图28.7给出了个性化探索中探索与利用的闭环权衡，RL Agent 根据用户兴趣确定性与系统不确定性在探索与利用之间动态权衡，用户反馈同时用于奖励回流与兴趣认知更新。

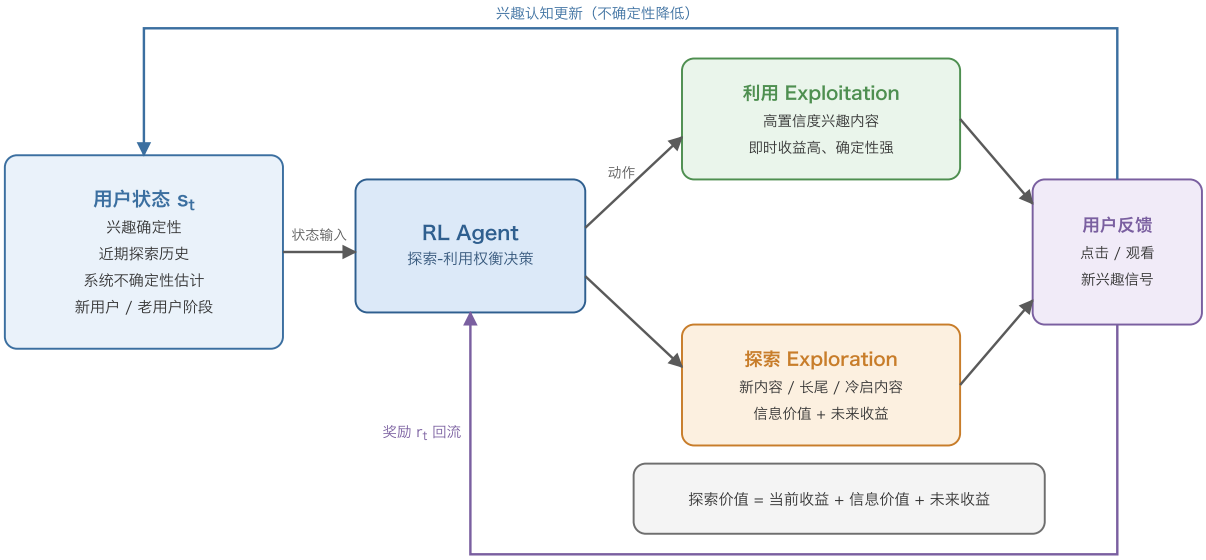


图 28.7: 个性化探索中探索与利用的闭环权衡。

对于新用户来说，增强探索性显得尤为重要。由于新用户的历史行为非常有限，传统兴趣建模通常很难获得高置信度的偏好表示，进而推荐系统整体的推荐精准度是非常差的。在这种情况下，如果一开始就完全按照模型当前的最大预测结果进行利用，很容易形成错误的兴趣锁定，导致新用户看不到自己真正喜欢看的内容，而无法在当前系统中形成稳定的留存。而通过有限的探索甚至加大对于新用户探索的力度，推荐系统就可以更加快速地获取到用户的偏好信息，从而降低后续推荐的不确定性。对于老用户来说，探索的目标通常与新用户的探索目标不同，更多的是对老用户的兴趣进行扩展、同时防止用户兴趣的收缩。例如，一个老用户长期只消费某一种内容类型，但其历史行为中已经隐含了一些潜在兴趣，那么适度探索相关内容可能使用户逐步形成新的兴趣路径。这类探索也更加接近长期用户价值优化，而不是单纯的短期流量分发。

因此，个性化探索的核心并不是简单地“增加随机性”，而是根据用户价值和系统不确定性主动决定什么时候探索、探索什么以及探索多少。这也是强化学习区别于传统随机探索规则的重要地方。同时，探索策略还需要区分不同的探索对象。可以探索某一个具体 Item，也可以探索某个内容垂类、作者、内容风格甚至整个业务频道。探索粒度越粗，策略空间通常越小，但获得的信息也越粗；探索粒度越细，策略更加灵活，但动作空间和数据需求都会显著增加。对于工业级推荐系统而言，探索还必须受到严格约束。探索流量不能无限增加，否则会直接损害线上用户体验。因此，实际策略通常需要在探索收益与即时业务收益之间进行动态平衡，并通过安全策略、流量限制和离线评估保证线上风险可控。

此外，探索策略还容易形成反馈闭环。某类内容因为策略给予更多探索而获得更多曝光，从而收集到更多正反馈，进一步导致未来得到更多流量；反过来，没有得到曝光的内容又因为缺少数据而继续无法获得曝光。这种“探索影响数据、数据再影响策略”的循环是工业推荐中的重要问题。因此，探索本身并不是一个孤立的 RL 问题，还与推荐系统中的曝光偏差、样本选择偏差以及冷启动问题密切相关。在后续第33章中，我们将对推荐系统中的探索与利用这一话题进行详细讨论，同时也会介绍强化学习推荐在探索与利用中的具体应用。

28.8 强化学习推荐讨论

从前几节讨论的强化学习推荐场景中我们可以看到，强化学习在工业推荐系统中的应用并不是单一形式，而是覆盖了从流量资源分配到多任务融合、从层次化决策到序列生成、再到长期用户价值优化等多个层次。如果

按照决策对象进行总结，流量资源分配更偏向于策略层决策，多任务融合更偏向于目标层决策，层次化决策强调不同粒度之间的策略协同，流量是否降级属于系统控制层决策，Session 优化强调跨请求长期价值，重排和混排则更多属于列表和序列决策，个性化探索则强调不确定性下的动态决策。这些问题表面上差异很大，但都具有一个共同特征，即：

笔记 传统的静态监督学习目标不足以完整表达真实的业务目标，需要通过强化学习等动态决策方法进行补充。

这些强化学习推荐场景虽然形式不同，但背后具有一些共同的特征。首先，推荐系统当前的动作会影响用户未来的状态。例如，当前曝光的内容会影响用户下一次请求的兴趣状态；当前是否执行降级会影响后续推荐系统的负载情况；当前是否增加探索会影响未来对用户兴趣的认知以及未来用户是否会陷入信息茧房等。其次，优化目标通常并非单一的即时 reward，而是存在多个目标之间的长期权衡。最后，很多决策具有明显的动态特性，使用固定规则通常很难适应不同的用户、不同的场景以及不同的系统状态。

因此，一个非常重要的判断标准是：**当前 action 是否会改变未来的状态分布**。如果当前动作只影响当前预测结果，而不会明显影响未来，那么使用复杂 RL 模型的收益可能就非常有限了。而如果当前动作会明显改变用户后续行为、内容供给情况、系统算力资源以及 RL Agent 未来的可选动作等，那么这个问题就非常符合强化学习的建模范式。当然，工业推荐中的状态通常并不是完全可观测的。用户真正的兴趣、疲劳程度、满意度以及未来是否继续使用系统等变量无法被直接观测，系统只能通过点击、观看、停留以及其他行为间接推断。因此，从严格意义上来看，大量工业推荐问题实际上更接近部分可观测马尔科夫决策过程（Partially Observed Markov Decision Process, POMDP）。在工程实现中通常采用用户历史序列、session 状态或者可学习的表征对隐藏状态进行刻画与近似。正是由于工业推荐问题的部分可观测性，使得策略需要在不完全信息下进行决策，这也是工业级 RL 与经典完全可观测环境中的 RL 的一个重要区别。我们在实际工程中也经常发现使用传统强化学习 TD-Learning 的方式学习出来的 Critic 网络通常预估的 Q 值和真实累计 reward 之间具有非常大的误差，存在 Q 值学习非常不准的问题。这通常也是工业级推荐噪声大、数据稀疏以及状态不完全可观测等多种因素所导致的。

此外，工业级推荐中的 RL action 也存在一个重要特点，即很多动作实际上并不是直接生成候选内容，而是改变后续推荐系统的搜索空间。例如，高层流量分配的 RL Agent 在决定某种内容的曝光比例之后，低层内容选择的 RL Agent 才能在当前内容的候选空间内选择具体的内容列表。这意味着工业级 RL 模型往往具有明显的层次结构和模块化特点，与经典游戏环境中的 RL Agent 可以直接控制最终动作的形式存在着非常大的差异。因此，我们可以将工业级推荐系统中的强化学习应用概括为一个从“即时预测”走向“连续决策”的过程。其中，传统推荐模型更加擅长回答：

“当前用户在当前请求中更可能喜欢什么？”

而强化学习模型更加关注：

“在当前状态下采取什么动作，才能让未来一段时间的累计价值最大？”

需要注意的是，上述这两类问题并不是相互替代的关系，而是一种互补的关系。工业级推荐系统不仅需要大量成熟的监督学习模型承担 CTR、CVR、时长、互动等目标的基础预估任务，还需要个性化流量调控、个性化多任务融合来保证推荐系统的优化效率。强化学习更多承担的是传统监督学习模型不容易处理的动态策略决策与长期目标优化这些建模任务。

进一步来看，工业级 RL 推荐系统与传统推荐系统之间更适合形成一种“预测模型 + 决策模型”的协同关系。预测模型负责告诉 Agent 当前每个候选可能产生什么即时结果；而 RL 模型则综合这些结果、用户状态、系统约束以及未来价值来进行最终的决策。这样才能够充分复用现有的工业级推荐系统，也能够避免针对所有问题都重新设计与构建一套 RL 架构的问题。因此，从推荐系统架构层面来看，一个较为典型的工业级 RL 推荐框架可以表示为图28.8所示的闭环结构：状态（用户兴趣、请求上下文与系统指标）输入 RL Agent，Agent 结合传统预测模型提供的预估信号输出动作，推荐系统作为环境执行曝光，用户反馈再以奖励回流和状态更新的形式驱动下一轮决策。

其中，传统推荐模型负责提供丰富的预测信号，RL Agent 则根据这些信号以及用户和系统状态进行决策，环境执行该决策后产生用户行为和系统反馈，并进一步形成下一时刻的状态。这样，传统推荐系统与强化学习策

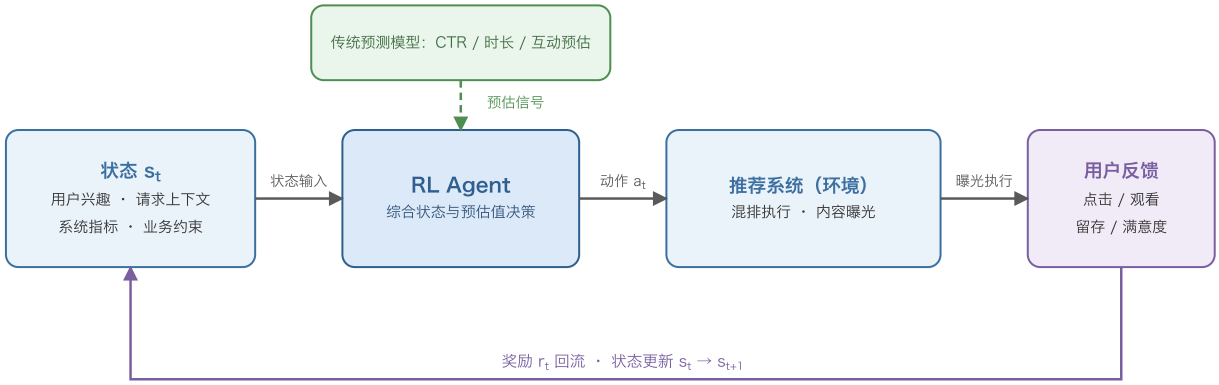


图 28.8: 工业级 RL 推荐系统的闭环框架。

略共同构成完整的闭环。如果进一步展开这个闭环，可以发现工业级 RL 模型其实包含如下四个相互关联的核心问题。

1. 第一是**状态表示**，即系统究竟应该使用哪些信息刻画当前状态；
2. 第二是**动作定义**，即 RL 真正应该控制什么；
3. 第三是**奖励设计**，即企业究竟希望策略优化什么；
4. 第四是**状态转移**，即当前动作如何影响用户和系统未来状态。这四个问题往往比具体采用哪一种 RL 算法更加重要。

在上述问题中，强化学习推荐中的动作定义需要进行合理的设计。以 Session 级 RL 建模为例，如果将动作定义为“直接选择一个物品”，那么动作空间就会达到数百万甚至更大的规模，因此为了便于 RL 系统落地，在工业实践中通常更可能让 RL Agent 来决定某类内容的曝光占比、排序公式中的调节项等。反之，如果 RL Agent 的动作定义得过于粗糙，比如只决定用户整体 session 中某类内容的曝光数量上限，那么虽然 RL 模型比较容易训练，但可能无法捕捉到足够细粒度的策略收益与策略差别。因此，强化学习推荐的动作定义需要在决策能力、动作空间和工程可执行性之间取得某种平衡。

此外，强化学习推荐的奖励设计同样也需要在不同维度之间取得平衡。具体来说，一个简单的即时 reward 可以很容易让 RL 模型较为稳定的训练，但有可能让 RL 模型重新退化为短期优化而无法捕捉到长期收益；相反，过于复杂的长期 reward 虽然更加符合业务目标，却可能带来稀疏反馈、长期信用分配（credit assignment）困难以及模型训练不稳定等多种问题。因此，工业级 RL 模型需要在即时奖励和长期奖励之间进行组合，并根据具体业务选择合适的时间尺度。这样才能够让模型在学习到长期收益的同时，能够相对稳定的训练与收敛。

28.9 强化学习推荐的挑战

工业级强化学习推荐与经典强化学习环境之间存在明显差异。推荐系统中的 RL Agent 动作空间通常非常大，用户状态具有高维、动态以及部分可观测等特点，同时奖励还存在稀疏、延迟和噪声大等问题。此外，线上过度探索不仅可能影响用户体验，还会带来实际的业务风险和系统故障风险。与经典强化学习不同，工业级推荐系统的大量训练数据通常来自已有推荐策略产生的历史日志，而不是由当前策略自由探索获得的，相当于推荐系统在不断“自我学习与迭代”的过程中持续产生数据。因此，离线数据利用、数据分布与策略偏移、策略评估与安全探索、长期信用分配、稳定性保障、大规模动作空间、低延迟推理等问题，都会成为 RL 模型实际落地中的关键难题。

首先，离线数据利用问题尤为重要。传统监督学习可以直接利用曝光日志中的数据进行训练，而强化学习需要进一步回答“如果采取另外一个 Action，会发生什么”。历史日志通常只记录了旧策略实际选择的 Action，因此数据无法完整覆盖所有可能的策略，这会带来典型的离线强化学习与分布外动作（Out-of-Distribution Action）问题。在工业实践中，通常需要设计专门的离线强化学习算法，或者通过策略约束、重要性采样等方法尽量缓解这一问题。此外，线上的 RL Agent 还需要面对探索问题，需要在控制风险的前提下，让动作空间中的更多动

作得到充分采样，从而持续产生可用于训练的数据。

其次是数据分布与策略漂移问题。推荐系统本身是一个持续迭代的系统，召回、排序、内容供给以及用户行为都会不断变化，因此历史数据并不一定始终适用于当前策略。如果 RL 策略长期依赖旧日志进行训练，就可能出现明显的数据分布漂移；反过来，RL 策略上线后产生的新数据又会进一步改变推荐系统中其他模型的训练分布，从而形成持续迭代的闭环。一旦 RL 模型出现 Action 数据分布异常漂移或 Action 极化坍缩，就可能导致推荐全链路训练数据分布发生较大波动，甚至引发系统性风险，最终造成大规模线上故障。因此，在工业级推荐系统中，强化学习模型的落地通常会格外保守和谨慎。

再次是策略评估与安全探索问题。推荐系统不能为了测试新的 RL 调控策略，而让大量真实用户承担潜在风险，因此通常需要先利用历史日志进行离线评估。然而，推荐系统中的 RL Action 会进一步影响未来状态，简单比较不同 RL Action 的历史平均 Reward 往往会产生严重偏差。因此，如何可靠地估计新策略可能带来的长期收益，是工业级 RL 模型能否真正上线的重要前提。另一方面，传统监督学习模型可以在固定数据上进行相对稳定的训练，而 RL 模型的有效学习通常依赖一定程度的探索。但在商业化推荐系统中，错误的探索会直接影响用户体验和业务收入。因此，线上探索通常不是完全随机的，而是需要限定探索流量、限制动作空间、设置安全边界，并通过逐级放量来保证系统的稳定。

此外，工业级推荐系统中的奖励通常具有明显的延迟性，单次 Action 的价值往往无法通过即时反馈直接判断。例如，一次推荐可能不会立即产生明显的用户反馈，但会影响用户后续的观看、互动甚至长期留存。因此，RL Agent 需要将较长时间范围内的多个行为反馈归因到之前的决策中，这带来了典型的长期信用分配（Long-term Credit Assignment）问题。尤其在推荐系统中，一次推荐行为可能经过多个推荐页面、多个内容消费以及较长时间后才体现最终价值，如何准确判断不同 Action 对长期 Reward 的贡献，是 RL 模型学习长期用户价值的关键。

同时，工业级推荐系统还需要高度重视稳定性保障（Stability）。推荐系统通常由召回、粗排、精排、重排等多个模块组成，RL Agent 的策略变化不仅会直接影响线上推荐结果，还可能进一步改变后续训练数据的分布，从而影响其他模型的迭代。如果策略更新过快或者出现异常，可能造成推荐结果、用户行为以及训练数据的连续波动，并通过系统闭环进一步放大。因此，工业部署通常需要配合策略版本管理、流量隔离、灰度发布、自动回滚、异常监控以及安全兜底等机制，将 RL Agent 的策略变化控制在可观测、可回滚的范围内。对于工业级推荐系统而言，RL 模型不仅需要“学得好”，还必须保证“上线后不出问题”。

最后，推荐系统还面临大规模动作空间与低延迟推理问题。经典 RL 环境中的动作空间通常相对有限，而推荐系统可能需要从数百、数千甚至更大的候选集合中进行决策。如果 RL 直接对所有候选进行策略搜索，推理成本可能难以接受。因此，工业实现通常需要借助候选过滤、层次化决策、动作分解或者已有排序模型等方式缩小策略搜索空间。

上述问题决定了工业级强化学习不可能仅仅按照经典教材中的“定义 MDP → 训练 RL Agent → 部署上线”这一流程简单完成。真正的工业级落地通常需要将 RL 框架嵌入已有的推荐系统架构中，通过成熟的预测模型、规则体系、监控系统、稳定性保障以及在线实验平台，共同构建完整的策略闭环。从这一角度来看，强化学习并不一定是推荐系统最底层、最核心的模型，而更像是为推荐系统提供一层策略优化能力。传统推荐模型已经解决了大量“内容理解”和“即时行为反馈预测”的问题，而强化学习进一步解决的是“如何动态利用这些预测结果进行决策”的问题。

这种定位也决定了工业级 RL 模型最有价值的方向，往往不是追求一个完全端到端的单一大模型，而是找到传统推荐架构中最需要动态策略能力的决策环节，并在这些环节中发挥作用。这也是本部分后续章节重点讨论的内容：本章主要回答“强化学习可以应用在哪里”的问题，而后续章节则进一步回答“这些问题应该如何用强化学习建模”的问题。

28.10 本章小结

本章从工业推荐系统的整体视角对强化学习的典型应用场景进行了系统梳理。与传统监督学习模型主要针对单次请求进行即时预测不同，强化学习更加适合处理当前决策会影响未来状态、目标具有长期性以及多个目标之间需要动态权衡的推荐问题。围绕这一核心区别，本章分别介绍了强化学习在工业推荐中的七类典型应用

场景，并结合两节讨论内容对强化学习的整体定位和落地挑战进行了归纳。

在具体应用场景层面，本章依次讨论了以下几类问题：流量分配与调控将不同内容类型、业务频道之间的流量分配转化为基于用户和系统状态的动态资源分配问题，能够在业务约束下实现比静态比例和分桶规则更精细的策略；流量降级优化将系统稳定性纳入决策目标，让强化学习在推荐质量与系统资源之间进行联合优化，并显式建模相邻请求之间的降级状态转移关系；多任务融合将多目标预估结果之上的融合策略视为一个独立的决策问题，通过强化学习动态调整各目标的权衡关系并优化融合公式中的超参数，与已有的多任务预估和 LTR 模型形成分工协同；Session 级推荐优化将用户在一个 session 内的连续请求建模为序列决策过程，使模型能够权衡即时价值与长期价值，主动应对连续内容消费带来的兴趣漂移和用户疲劳等问题；层次化决策将复杂推荐问题拆分为不同粒度、不同频率的高层和低层策略，在缩小低层动作空间的同时更自然地承载业务约束；序列生成优化将重排和混排阶段的列表构造抽象为逐步决策过程，直接针对列表级和序列级指标进行优化，并可以在工程上通过对序列生成策略的超参数进行控制来降低动作空间的复杂度；个性化探索则将探索与利用之间的权衡从固定规则升级为依赖用户状态和系统不确定性的动态策略，兼顾新用户兴趣发现、老用户兴趣扩展以及冷启动与长尾内容的曝光机会。

在此基础上，本章进一步对强化学习推荐进行了整体讨论。这些看似形态各异的应用场景可以按照强化学习所控制的决策粒度进行统一理解：流量调控和降级更接近系统与策略层决策，多任务融合更加偏向目标层决策，层次化决策解决的是不同粒度 Agent 之间的协同问题，Session 优化关注跨请求的长期价值，序列生成关注列表和序列级决策，而个性化探索则主要处理不确定性条件下的动态策略选择。判断一个推荐问题是否适合使用强化学习的关键标准是：**当前 Action 是否会明显改变未来的状态分布**。如果当前动作只影响即时预测，那么继续使用监督学习通常已经足够；如果当前动作会影响用户后续行为、内容供给、系统资源或未来可选动作，那么这个问题就更符合强化学习的建模范式。也正因如此，工业级推荐系统中大量问题实际上更接近部分可观测的 POMDP，需要通过用户历史序列、session 状态或可学习表征对隐藏状态进行近似。最终，一个较为合理的工业级架构是让传统监督学习模型继续承担 CTR、CVR、时长等即时预估任务，让强化学习模型在这些预测信号之上进行“连续决策”与长期价值优化，从而形成“预测模型 + 决策模型”的协同关系。

本章最后专门讨论了强化学习推荐从理论走向工业落地过程中面临的关键挑战，包括：离线数据利用与分布外动作问题、数据分布漂移与推荐系统闭环带来的策略偏移风险、离线策略评估与安全探索之间的平衡、奖励延迟与长期信用分配问题、由多模块耦合和策略闭环放大所引发的稳定性风险，以及大规模动作空间下的低延迟推理约束。这些挑战决定了工业级强化学习并不能简单遵循“定义 MDP → 训练 RL Agent → 部署上线”的经典流程，而必须嵌入到已有的推荐系统架构中，并结合成熟的预测模型、规则体系、监控与安全兜底机制共同构成完整的策略闭环。

综合来看，本章希望回答的核心问题是“强化学习在工业推荐系统中可以应用在哪里”，即在整个推荐链路中识别出那些传统监督学习难以充分刻画的动态决策环节。在此基础上，后续章节将进一步回答“这些问题应该如何用强化学习进行具体建模”的问题，围绕 MDP 与 POMDP 建模、状态与动作空间设计、奖励函数构造、长期价值建模、离线强化学习、策略优化、探索与利用以及工业线上部署等内容展开，并结合本章介绍的各类应用场景讨论强化学习从理论走向真实工业系统时的具体做法与工程细节。